Les bases de données

Stocker et retrouver des informations, expliqué simplement

1. C'est quoi une base de données ?

Une **base de données** est un ensemble d'informations **organisées** et stockées de façon à pouvoir être retrouvées, ajoutées, modifiées et supprimées facilement. On en utilise tous les jours sans le voir : les comptes d'un site, les élèves d'un lycée, les produits d'une boutique en ligne, les contacts d'un téléphone.

> **Analogie :** imagine un grand classeur contenant plusieurs feuilles de tableur (type Excel) reliées entre elles. Chaque feuille range un type d'information (les élèves, les classes...). La base de données fait cela, mais avec des milliers ou des millions de lignes, plusieurs utilisateurs en même temps et des règles pour éviter les erreurs.

2. Le SGBD

Le logiciel qui crée et gère la base s'appelle un **SGBD** (Système de Gestion de Base de Données). On lui parle avec un langage : le **SQL**.

SGBD	À savoir
MySQL / MariaDB	Très répandu, gratuit, utilisé avec PHP pour les sites web
PostgreSQL	Gratuit, puissant, très complet
SQLite	Une base dans un simple fichier, sans serveur (applications mobiles, petits projets)
SQL Server	Produit de Microsoft, souvent utilisé avec C# et .NET
Oracle	Grandes entreprises
MongoDB	Base **NoSQL** (pas de tables), pour des données moins structurées

Dans ce document, on parle des bases **relationnelles** : les données sont rangées dans des **tables**.

3. Le vocabulaire : table, ligne, colonne

Voici une table `eleve`. Elle contient 3 élèves :

id	nom	prenom	age	id_classe
1	Martin	Léa	19	1
2	Durand	Hugo	20	2
3	Bernard	Sara	18	1

Terme	Signification	Exemple
Table	Un ensemble de données sur un même sujet	`eleve`
Colonne (champ)	Une information, avec un nom et un type	`nom`, `age`
Ligne (enregistrement)	Un élément complet de la table	La ligne de Léa Martin
Clé primaire	Colonne dont la valeur est **unique** pour chaque ligne : elle identifie la ligne	`id`
Clé étrangère	Colonne qui renvoie vers la clé primaire d'une **autre** table	`id_classe`

4. Relier les tables

Plutôt que d'écrire le nom de la classe sur chaque ligne d'élève (on le répéterait des centaines de fois), on crée **une table** `classe` et on y fait référence avec la clé étrangère `id_classe` :

Table classe : id	nom
1	SIO1
2	SIO2

Dans `eleve`, Léa a `id_classe = 1` : elle est donc en **SIO1**. Si le nom de la classe change, on le modifie à **un seul endroit**. C'est le principe de base : **ne pas répéter les informations**.

Type de relation	Exemple
Un à plusieurs (1-N)	Une classe a plusieurs élèves ; un élève n'a qu'une classe
Plusieurs à plusieurs (N-N)	Un élève suit plusieurs options, une option a plusieurs élèves : on crée une table intermédiaire

5. Le SQL : les commandes de base

SQL (*Structured Query Language*) est le langage pour parler à une base relationnelle. D'abord, créer les tables :

```
CREATE TABLE classe (
  id  INT PRIMARY KEY,
  nom VARCHAR(20)
);

CREATE TABLE eleve (
  id        INT PRIMARY KEY,
  nom       VARCHAR(50),
  prenom    VARCHAR(50),
  age       INT,
  id_classe INT,
  FOREIGN KEY (id_classe) REFERENCES classe(id)
);
```

Commande	Action
`SELECT`	**Lire** des données
`INSERT`	**Ajouter** une ligne
`UPDATE`	**Modifier** des lignes
`DELETE`	**Supprimer** des lignes

```
-- Ajouter un élève
INSERT INTO eleve (id, nom, prenom, age, id_classe)
VALUES (1, 'Martin', 'Léa', 19, 1);

-- Lire tous les élèves
SELECT * FROM eleve;

-- Lire certaines colonnes, avec une condition et un tri
SELECT nom, prenom FROM eleve WHERE age > 18 ORDER BY nom;

-- Modifier
UPDATE eleve SET age = 20 WHERE id = 1;

-- Supprimer
DELETE FROM eleve WHERE id = 3;
```

> **Attention :** avec `UPDATE` et `DELETE`, n'oublie jamais le `WHERE`. Sans lui, la commande s'applique à **toutes** les lignes de la table !

6. Aller un peu plus loin avec SELECT

```
-- Compter les élèves
SELECT COUNT(*) FROM eleve;

-- Moyenne d'âge
SELECT AVG(age) FROM eleve;

-- Plusieurs conditions
SELECT * FROM eleve WHERE age >= 19 AND id_classe = 1;

-- Afficher le nom de la classe avec chaque élève (jointure)
SELECT eleve.nom, eleve.prenom, classe.nom
FROM eleve
JOIN classe ON eleve.id_classe = classe.id;
```

La **jointure** (`JOIN`) relie deux tables grâce à la clé étrangère : on obtient une ligne par élève avec le nom de sa classe.

7. Bonnes pratiques

• Chaque table a une **clé primaire**.

• **Ne pas répéter** les mêmes informations : on crée une autre table et on la relie par une clé étrangère.

• Choisir un **bon type** pour chaque colonne (nombre entier, texte, date...).

• Toujours mettre un **WHERE** avec `UPDATE` et `DELETE`.

• Faire des **sauvegardes** régulières de la base.

• Protéger l'accès : mots de passe forts et **droits limités** pour chaque utilisateur.

8. À retenir

• Une base de données **range des informations** de façon organisée ; le **SGBD** la gère.

• Dans une base relationnelle, les données sont dans des **tables** (colonnes + lignes).

• **Clé primaire** = identifiant unique ; **clé étrangère** = lien vers une autre table.

• Le **SQL** : `SELECT` (lire), `INSERT` (ajouter), `UPDATE` (modifier), `DELETE` (supprimer).

• `WHERE` filtre, `ORDER BY` trie, `JOIN` relie deux tables.